LAPORAN PRAKTIKUM

STRUKTUR DATA

“Queue”

disusun Oleh:

Sofian Arba’i

2511533029

Dosen Pengampu:

Dr. Wahyudi, S.T, M.T

Asisten Praktikum:

Sherly Sukmadira

DEPARTEMEN INFORMATIKA

FAKULTAS TEKNOLOGI INFORMASI

UNIVERSITAS ANDALAS

2026

KATA PENGANTAR

Assalamu'alaikum Warahmatullahi Wabarakatuh

Puji syukur kehadirat Allah SWT karena atas rahmat dan karunia-Nya, penulis dapat menyelesaikan Laporan Praktikum pekan 4 dengan judul "Queue" ini dengan baik. Laporan ini disusun sebagai salah satu bentuk pertanggungjawaban kegiatan praktikum serta sebagai sarana untuk meningkatkan pemahaman mengenai penggunaan struktur data Queue.

Dalam praktikum pekan 4 mempelajari dasar-dasar Queue, yaitu salah satu struktur data yang menggunakan prinsip FIFO (First In First Out), di mana elemen yang pertama dimasukkan akan menjadi elemen pertama yang dikeluarkan. Queue memiliki aturan khusus dalam proses penambahan dan penghapusan elemen. Selain itu, praktikum ini juga mempelajari berbagai operasi yang dapat dilakukan pada Queue, seperti enqueue, dequeue, dan peek.

Penyusunan laporan ini tidak terlepas dari bantuan, bimbingan, dan dukungan berbagai pihak. Oleh karena itu, saya ingin mengucapkan terima kasih yang sebesar-besarnya kepada:

1. Dosen pengampu mata kuliah Praktikum Struktur Data yang telah memberikan ilmu, arahan, serta bimbingannya.
2. Asisten praktikum yang telah membimbing dan memberikan penjelasan selama kegiatan praktikum berlangsung.

Saya meminta maaf jika masih terdapat kesalahan pada penulisan laporan ini. Oleh karena itu, saya menerima kritik dan saran yang membangun demi perbaikan di masa mendatang.

Padang, 27 April 2026

Sofian Arba'i

DAFTAR ISI

KATA PENGANTAR i
DAFTAR ISI ii
BAB I PENDAHULUAN 1
1.1 Latar Belakang 1
1.2 Tujuan 1
1.3 Manfaat 2
BAB II PEMBAHASAN 3
2.1 Pengertian Queue 3
2.1.1 Kelebihan dan Kekurangan Queue 4
2.2 Implementasi Queue pada Java 4
2.2.1 Class QueueArray_2511533029 5
2.2.2 Class IterasiQueue_2511533029 8
2.2.3 Class QueueLinkedList_2511533029 9
2.2.4 Class ReverseData_2511533029 10
BAB III PENUTUP 13
3.1 Kesimpulan 13
3.2 Saran 13
DAFTAR PUSTAKA 14

BAB I
PENDAHULUAN

1.1 Latar Belakang

Teknologi informasi kini berkembang sangat cepat, sehingga kita butuh cara mengelola data yang rapi dan efisien. Dalam pemrograman, jika data dikelola dengan baik, program akan berjalan lebih lancar dan lebih mudah dikembangkan di masa mendatang. Salah satu cara untuk mengatur data tersebut adalah dengan menggunakan struktur data bernama Queue.

Queue bekerja dengan prinsip sederhana: "siapa yang datang duluan, dilayani duluan" (First In First Out / FIFO). Artinya, data yang pertama kali dimasukkan ke dalam Queue akan menjadi data pertama yang diambil atau diproses. Berbeda dengan struktur data lain yang lebih bebas, Queue memiliki aturan yang ketat. Justru karena aturannya yang jelas ini, Queue sangat cocok digunakan untuk situasi seperti antrean di loket, penjadwalan tugas pada komputer, atau penyimpanan sementara data (buffer) dalam program.

Karena kegunaannya yang luas, penting bagi mahasiswa terutama yang belajar pemrograman dan struktur data untuk memahami cara kerja Queue. Melalui praktikum ini, diharapkan tidak hanya mengerti teori dasarnya, tetapi juga mampu menerapkan Queue untuk menyelesaikan berbagai masalah pemrograman secara praktis.

1.2 Tujuan

Tujuan dari pelaksanaan praktikum pekan 4 tentang Queue adalah sebagai berikut:

1. Mempelajari cara kerja struktur data Queue dalam pengelolaan data.
2. Mengembangkan kemampuan dalam merancang program menggunakan konsep Queue.

3. Menganalisis penerapan Queue untuk menyelesaikan berbagai kasus pemrograman secara efektif.
4. Memahami operasi dasar pada Queue, yaitu menambahkan data (*enqueue*), menghapus data (*dequeue*), dan melihat data paling depan tanpa menghapusnya (*peek*).

1.3 Manfaat

Manfaat yang diharapkan dari pelaksanaan praktikum pekan 4 tentang Queue adalah sebagai berikut:

1. Memberikan pemahaman tentang penerapan Queue dalam kasus nyata di pemrograman.
2. Meningkatkan keterampilan mahasiswa dalam merancang program yang menerapkan konsep Queue secara terstruktur dan efisien.
3. Membantu mahasiswa memahami dan menganalisis penerapan Queue untuk menyelesaikan berbagai kasus pemrograman secara efektif.
4. Memberikan pemahaman kepada mahasiswa mengenai operasi dasar Queue, seperti enqueue, dequeue, dan peek, dalam pengelolaan data pada program.

BAB II
PEMBAHASAN

2.1 Pengertian Queue

Queue atau dalam bahasa Indonesia yang berarti antrean adalah struktur data yang menyusun elemen-elemen data dalam urutan linier. Prinsip dasar dari struktur data ini adalah "First In, First Out" (FIFO) yang berarti elemen data yang pertama dimasukkan ke dalam antrean akan menjadi yang pertama pula untuk dikeluarkan.

Caranya bekerja adalah seperti jejeran orang yang sedang menunggu antrean di supermarket di mana orang pertama yang datang adalah yang pertama dilayani (First In, First Out). Pada struktur data ini, urutan pertama (data yang akan dikeluarkan) disebut Front atau Head. Sebaliknya, data pada urutan terakhir (data yang baru saja ditambahkan) disebut Back, Rear, atau Tail. Proses untuk menambahkan data pada antrean disebut dengan Enqueue, sedangkan proses untuk menghapus data dari antrean disebut dengan Dequeue.

Queue memiliki peran yang penting dalam berbagai aplikasi dan algoritma. Salah satu fungsi utamanya adalah mengatur dan mengelola antrean tugas atau operasi secara efisien. Dalam sistem komputasi, ia digunakan untuk menangani tugas-tugas seperti penjadwalan proses, antrean pesan, dan manajemen sumber daya.

Berikut beberapa jenis queue:

1. Berdasarkan Implementasinya

   inear/Simple Queue: Elemen-elemen data disusun dalam barisan linear dan penambahan serta penghapusan elemen hanya terjadi pada dua ujung barisan tersebut. Selain itu, Circular Queue: Mirip dengan jenis

linear, tetapi ujung-ujung barisan terhubung satu sama lain, menciptakan struktur antrean yang berputar.

2. Berdasarkan Penggunaan

Priority Queue: Setiap elemen memiliki prioritas tertentu. Elemen dengan prioritas tertinggi akan diambil terlebih dahulu dan Double-ended Queue (Dequeue): Elemen dapat ditambahkan atau dihapus dari kedua ujung antrean.

2.1.1 Kelebihan dan Kekurangan Queue

Meskipun struktur data queue memiliki banyak kegunaan, berikut beberapa kelebihan dan kekurangan yang perlu diperhatikan sebelum menggunakan queue.

Kelebihan:

1. Data berjumlah besar dapat dikelola dengan mudah dan efisien.
2. Proses insert dan delete data dapat dilakukan dengan mudah karena mengikuti aturan first in first out (FIFO).
3. Efisien dalam menangani tugas berdasarkan urutan kedatangan.
4. Queue dapat digunakan dalam implementasi struktur data lainnya.

Kekurangan:

1. Tidak efisien untuk pencarian elemen tertentu dalam antrean.
2. Memerlukan alokasi memori yang cukup untuk menyimpan antrean.
3. Operasi seperti penyisipan dan penghapusan elemen dari tengah cenderung banyak memakan waktu.

2.2 Implementasi Queue pada Java

Pada praktikum pekan 4 mahasiswa diminta untuk mengikuti intruksi dari dosen, yaitu menuliskan kembali kode program yang sudah ada di layar

dan memahami kode program tersebut. Kode program tentang queue array, iterasi queue, queue linkedlist, dan reverse data, berikut di bawah ini kode program yang dituliskan kembali ke dalam java.

2.2.1 Class QueueArray_2511533029

Kode program pertama yaitu kode program implementasi penggunaan queue dalam pemrograman java. Kode program ini menunjukkan cara mengimplementasikan struktur data Queue menggunakan array serta memperlihatkan proses penambahan, penghapusan, dan penampilan data sesuai prinsip FIFO tanpa mengubah urutan dasar antrian. Berikut kode program queue array:

```java
package pekan4_2511533029;

public class QueueArray_2511533029 {
    int front_3029, rear_3029, size_3029;
    int capacity_3029;
    int array_3029[];

    public QueueArray_2511533029 (int capacity_3029) {
        this.capacity_3029 = capacity_3029;
        front_3029 = this.size_3029 = 0;
        rear_3029 = capacity_3029 - 1 ;
        array_3029 = new int [this.capacity_3029];
    }

    boolean isFull_2511533029 (QueueArray_2511533029 queue) {
        return (queue.size_3029 == queue.capacity_3029);
    }

    boolean isEmpty_2511533029 (QueueArray_2511533029 queue) {
        return (queue.size_3029 == 0);
    }
    void enqueue_2511533029 (int item_3029) {
        if  (isFull_2511533029(this))
            return;
        this.rear_3029 = (this.rear_3029 + 1) % this.capacity_3029;
        this.array_3029 [this.rear_3029] = item_3029;
        this.size_3029 = this.size_3029 + 1;
        System.out.println(item_3029 + " enqueued to queue");
    }

    int dequeue_3029() {
        if (isEmpty_2511533029(this))
            return Integer.MIN_VALUE;
        int item_3029 = this.array_3029[this.front_3029];
        this.front_3029 = (this.front_3029 + 1) % this.capacity_3029;
        this.size_3029 = this.size_3029 - 1;
        return item_3029;
    }
```

Kode Program 2.1

```java
    int front_3029 () {
        if (isEmpty_2511533029(this))
            return Integer.MIN_VALUE;

        return this.array_3029[this.front_3029];
    }
    int rear_3029 () {
        if (isEmpty_2511533029(this))
            return Integer.MIN_VALUE;
        return this.array_3029[this.rear_3029];
    }
       //mencetak elemen antrian
    void display_3029() {
        int i_3029;
        if (front_3029 == rear_3029) {
            System.out.println("\n antrian kosong\n");
        }
        // kunjungidari belakang dan cetak
        for (i_3029 = front_3029; i_3029 < rear_3029; i_3029++) {
            System.out.printf(" %d <-- ", array_3029[i_3029]);
        }
        return;
    }
}
```

Kode Program 2.2

Kode program di atas merupakan implementasi struktur data Queue menggunakan array dalam bahasa Java melalui class QueueArray_2511533029. Class ini memiliki beberapa atribut utama, yaitu front_3029 sebagai penunjuk elemen paling depan, rear_3029 sebagai penunjuk elemen paling belakang, size_3029 sebagai jumlah elemen dalam queue, capacity_3029 sebagai kapasitas maksimum queue, serta array_3029[] sebagai media penyimpanan data. Konstruktor QueueArray_2511533029() digunakan untuk menginisialisasi kapasitas queue, mengatur nilai awal front, rear, dan size, serta membuat array sesuai kapasitas yang ditentukan. Implementasi ini menggunakan konsep circular array agar penggunaan ruang pada array menjadi lebih efisien.

Class QueueArray_2511533029 menyediakan beberapa method utama untuk mengelola queue. Method isFull_2511533029() digunakan untuk memeriksa apakah queue sudah penuh, sedangkan isEmpty_2511533029() digunakan untuk memeriksa apakah queue kosong. Method enqueue_2511533029() berfungsi menambahkan data ke bagian belakang queue, sedangkan dequeue_3029() berfungsi

menghapus data dari bagian depan queue sesuai prinsip FIFO (First In First Out). Selain itu, method front_3029() digunakan untuk melihat elemen paling depan, method rear_3029() untuk melihat elemen paling belakang, dan method display_3029() untuk menampilkan isi queue. Method-method tersebut menunjukkan operasi dasar Queue, yaitu enqueue, dequeue, dan peek.

Untuk menjalankan kode program 2.1 dan 2.2 dibutuhkan suatu kode program lagi dengan class berbeda. Kode program ini disebut sebagai *driver* atau penguji, yaitu menjalankan program untuk mencoba dan menampilkan hasil dari setiap operasi yang telah disediakan pada class QueueArray_2511533029. Berikut kode program dari class driver QueueArrayDriver_2511533029:

```
package pekan4_2511533029;

public class QueueArrayDriver_2511533029 {
    public static void main (String [] args) {
            QueueArray_2511533029 queue_3029 = new QueueArray_2511533029 (1000);
            queue_3029.enqueue_2511533029(10);
            queue_3029.enqueue_2511533029(20);
            queue_3029.enqueue_2511533029(30);
            queue_3029.enqueue_2511533029(40);
            System.out.println("item di depan " + queue_3029.front_3029());
            System.out.println("item paling belakang " + queue_3029.rear_3029());
            System.out.println("tampilan queue ");
            queue_3029.display_3029();
            System.out.println();
            System.out.println(queue_3029.dequeue_3029() + " dihapus dari queue");
            System.out.println("item di depan: " + queue_3029.front_3029());
            System.out.println("item di belakang: " + queue_3029.rear_3029());
            System.out.println("tampilan queue setelah satu data dihapus");
            queue_3029.display_3029();
    }

}
```

Kode Program 2.3

Pada class QueueArrayDriver_2511533029, program utama digunakan untuk menguji implementasi Queue dengan membuat objek queue berkapasitas 1000. Program kemudian menambahkan beberapa data, yaitu 10, 20, 30, dan 40 menggunakan method enqueue, lalu menampilkan elemen paling depan, elemen paling belakang, dan isi queue. Setelah itu, satu elemen dihapus menggunakan method dequeue,

kemudian program kembali menampilkan kondisi queue setelah penghapusan.

2.2.2 Class IterasiQueue_2511533029

Kode program selanjutnya yaitu kode program yang menunjukkan cara menyimpan dan menampilkan data pada Queue menggunakan LinkedList, serta memperlihatkan proses penelusuran isi antrian secara berurutan tanpa menghapus elemen yang ada. Berikut kode program iterasi queue:

```java
package pekan4_2511533029;

import java.util.Iterator;
import java.util.Queue;
import java.util.LinkedList;

public class IterasiQueue_2511533029 {

    public static void main(String[] args) {
        Queue <String> q_3029 = new LinkedList<> ();

        q_3029.add("Praktikum");
        q_3029.add("Struktur");
        q_3029.add("Data");
        q_3029.add("Dan");
        q_3029.add("Algoritma");
        Iterator <String> iterator_3029 = q_3029.iterator();
        while (iterator_3029.hasNext()) {
            System.out.print(iterator_3029.next() + " ");
        }

    }

}
```

Kode Program 2.4

Kode program 2.4 merupakan kode program implementasi sederhana Queue menggunakan LinkedList pada Java melalui class IterasiQueue_2511533029. Pada program ini, interface Queue<String> digunakan untuk menyimpan data bertipe String, sedangkan LinkedList<> digunakan sebagai implementasinya karena mendukung operasi antrian secara dinamis. Di dalam method main, program membuat objek queue bernama q_3029, kemudian menambahkan beberapa elemen berupa teks, yaitu “Praktikum”, “Struktur”, “Data”,

"Dan", dan "Algoritma" menggunakan method add(). Setelah itu, program menggunakan Iterator<String> untuk melakukan iterasi terhadap seluruh elemen queue secara berurutan dari depan ke belakang. Proses iterasi dilakukan dengan perulangan while dan method hasNext() untuk memeriksa apakah masih ada elemen berikutnya, serta next() untuk mengambil dan menampilkan elemen tersebut.

2.2.3 Class QueueLinkedList_2511533029

Kode program queue linkedlist merupakan kode program yang menunjukkan penggunaan struktur data Queue dengan implementasi LinkedList pada Java, mulai dari proses penambahan elemen, penghapusan elemen terdepan, melihat kepala antrian, hingga menghitung jumlah elemen. Berikut kode program queue linkedlist:

```java
package pekan4_2511533029;

import java.util.Queue;
import java.util.LinkedList;

public class QueueLinkedList_2511533029 {

    public static void main(String[] args) {
        Queue<Integer> q_3029 = new LinkedList <> ();
        // tambah elemen {0,1,2,3,4,5} ke antrian
        for (int i=0; i<=6; i++)
            q_3029.add(i);
        // menampilkan isi antrian.
        System.out.println("Elemen Antrian: " + q_3029);
        // Untuk menghapus kepala antrian.
        int hapus_3029 = q_3029.remove();
        System.out.println("Hapus Elemen = " + hapus_3029);
        System.out.println(q_3029);
        // Untuk melihat antrian terdepan
        int depan_3029 = q_3029.peek();
        System.out.println("Kepala Antrian = " + depan_3029);

        int banyak_3029 = q_3029.size();
        System.out.println("Size Antrian = " + banyak_3029);
    }

}
```

Kode Program 2.5

Kode program di atas merupakan implementasi struktur data Queue menggunakan LinkedList pada Java melalui class QueueLinkedList_2511533029. Pada program ini, interface

Queue<Integer> digunakan untuk menyimpan data bertipe bilangan bulat, sedangkan LinkedList<> digunakan sebagai implementasi queue karena dapat mengelola data secara dinamis. Di dalam method main, program membuat objek queue bernama q_3029, kemudian menambahkan elemen angka dari 0 sampai 6 ke dalam antrian menggunakan perulangan for dan method add(). Setelah seluruh elemen dimasukkan, program menampilkan isi queue untuk menunjukkan urutan data yang tersimpan sesuai prinsip FIFO (First In First Out).

Selanjutnya, program melakukan beberapa operasi dasar pada queue. Method remove() digunakan untuk menghapus elemen paling depan (kepala antrian), kemudian nilai yang dihapus disimpan pada variabel hapus_3029 dan ditampilkan. Setelah penghapusan, isi queue ditampilkan kembali untuk memperlihatkan perubahan data. Program juga menggunakan method peek() untuk melihat elemen paling depan tanpa menghapusnya, lalu hasilnya disimpan pada variabel depan_3029. Selain itu, method size() digunakan untuk menghitung jumlah elemen yang tersisa di dalam queue dan hasilnya disimpan pada variabel banyak_3029.

2.2.4 Class ReverseData_2511533029

Kode program terakhir pada pekan 4 praktikum yaitu kode program yang menunjukkan cara membalik urutan elemen pada Queue dengan memanfaatkan Stack sebagai struktur bantu, sehingga data yang semula berurutan dari depan ke belakang dapat dibalik tanpa mengubah nilai elemennya. Berikut kode program reverse data:

```java
package pekan4_2511533029;

import java.util.LinkedList;
import java.util.Queue;
import java.util.Stack;

public class ReverseData_2511533029 {

    public static void main(String[] args) {
        Queue <Integer> q_3029 = new LinkedList <Integer> ();
        q_3029.add(1);
        q_3029.add(2);
        q_3029.add(3); // [1,2,3]
        System.out.println("sebelum reverse: " + q_3029);
        Stack <Integer> s_3029 = new Stack <Integer> ();
        while (!q_3029.isEmpty()) { // Q -> S
            s_3029.push (q_3029.remove());
        }
        while (!s_3029.isEmpty()) { // S -> Q
            q_3029.add (s_3029.pop());
        }
        System.out.println("sesudah reverse: " + q_3029); // [3,2,1]
    }

}
```

Kode Program 2.6

Kode program 2.6 merupakan implementasi pembalikan urutan data pada Queue menggunakan bantuan Stack dalam Java melalui class ReverseData_2511533029. Pada program ini, interface Queue<Integer> diimplementasikan menggunakan LinkedList untuk menyimpan data antrian, sedangkan Stack<Integer> digunakan sebagai struktur bantu untuk membalik urutan elemen. Di dalam method main, program terlebih dahulu membuat objek queue q_3029 dan menambahkan tiga elemen, yaitu 1, 2, dan 3, sehingga isi awal queue adalah [1, 2, 3]. Kondisi awal ini kemudian ditampilkan melalui perintah System.out.println() untuk menunjukkan urutan data sebelum proses pembalikan dilakukan.

Proses pembalikan data dilakukan dalam dua tahap utama. Tahap pertama, seluruh elemen pada queue dipindahkan ke stack menggunakan perulangan while (!q_3029.isEmpty()), di mana setiap elemen dihapus dari queue dengan method remove() lalu dimasukkan ke stack menggunakan method push(). Proses ini memanfaatkan sifat stack yang bekerja dengan prinsip LIFO (Last In First Out), sehingga urutan data akan terbalik saat disimpan. Tahap kedua, seluruh elemen

pada stack dipindahkan kembali ke queue menggunakan perulangan while (!s_3029.isEmpty()), yaitu dengan mengambil elemen teratas stack menggunakan method pop() lalu menambahkannya kembali ke queue menggunakan method add().

BAB III
PENUTUP

3.1 Kesimpulan

Berdasarkan praktikum pekan 4, dapat disimpulkan bahwa Queue merupakan struktur data linear yang bekerja dengan prinsip FIFO (First In First Out), yaitu data yang pertama masuk akan menjadi data pertama yang keluar. Melalui praktikum ini, mahasiswa dapat memahami konsep dasar Queue, mulai dari cara kerja, fungsi, hingga penerapannya dan operasi dasar seperti enqueue, dequeue, dan peek, serta penerapannya dalam pemrograman Java menggunakan array, LinkedList, dan stack. Dari beberapa implementasi program yang telah dilakukan, dapat dilihat bahwa Queue berperan penting dalam pengelolaan data secara teratur dan efisien. Dengan demikian, praktikum ini berhasil meningkatkan pemahaman mahasiswa mengenai konsep Queue dan penerapannya dalam menyelesaikan permasalahan pemrograman.

3.2 Saran

Pada praktikum berikutnya, disarankan agar mahasiswa tidak hanya menuliskan ulang kode program yang diberikan, tetapi juga mencoba memodifikasi dan mengembangkan program secara mandiri agar pemahaman terhadap konsep Queue menjadi lebih mendalam. Mahasiswa juga perlu lebih teliti dalam memahami alur program, khususnya pada penggunaan operasi dasar seperti enqueue, dequeue, dan peek, sehingga dapat mengetahui fungsi setiap bagian kode dengan lebih baik. Selain itu, disarankan agar mahasiswa terus berlatih mengimplementasikan Queue dalam berbagai kasus pemrograman yang lebih kompleks dan bervariasi

DAFTAR PUSTAKA

[1] Rizki Maulana, “Struktur Data Queue: Pengertian, Fungsi, dan Jenisnya,”. [Daring]. Tersedia pada: **Struktur Data Queue: Pengertian, Fungsi, dan Jenisnya** [Diakses: 27-Apr-2026].

[2] Trivusi, “Struktur Data Queue: Pengertian, Jenis, dan Kegunaannya,”. [Daring]. Tersedia pada: **Struktur Data Queue: Pengertian, Jenis, dan Kegunaannya - Trivusi** [Diakses: 27-Apr-2026].

LAPORAN TUGAS PRAKTIKUM

Struktur Data

Pekan 4 (Queue)

Dosen Pengampu:

Dr. Wahyudi, S.T, M.T

Asisten Praktikum:

Sherly Sukmadira

DEPARTEMEN INFORMATIKA

FAKULTAS TEKNOLOGI INFORMASI

UNIVERSITAS ANDALAS

2026

TUGAS QUEUE

1. Deskripsi Tugas

Mahasiswa diminta untuk mengimplementasikan struktur data Queue (Antrian) menggunakan array dalam bahasa Java. Studi kasus yang digunakan adalah antrian pelanggan di loket pelayanan. Setiap pelanggan yang datang akan masuk ke dalam antrian dan akan dilayani berdasarkan prinsip FIFO (First In First Out).

Program harus mampu:

- Menambahkan pelanggan ke antrian (**enqueue**)
- Menghapus pelanggan dari antrian (**dequeue**)
- Menampilkan isi antrian
- Mengecek kondisi antrian (kosong atau penuh)

2. Aturan Penamaan (Wajib Dipatuhi)

1. Nama Kelas

   Gunakan akhiran NIM lengkap Contoh:

   - AntrianLoket_2311532008
   - Queue_2311532008
2. Nama File Java

   Harus sama dengan nama class

   Contoh: AntrianLoket_2311532008.java
3. Penamaan Variabel

   Gunakan standar berikut:

   - front → indeks depan
   - rear → indeks belakang
   - max → kapasitas array
   - queue[] → array penyimpan data

3. Struktur Tugas

Program minimal memiliki:

a. Deklarasi dan Inisialisasi

- Array untuk menyimpan data antrian
- Variabel front dan rear
- Kapasitas maksimal queue

b. Method yang Wajib Ada

1. enqueue()
   - Menambahkan data ke belakang antrian
   - Jika penuh → tampilkan pesan
2. dequeue()
   - Menghapus data dari depan antrian
   - Jika kosong → tampilkan pesan
3. display()
   - Menampilkan seluruh isi antrian
4. isEmpty()
   - Mengembalikan nilai true jika kosong
5. isFull()
   - Mengembalikan nilai true jika penuh
6. Reverse()
   - Membalik seluruh isi antrian

c. Menu Program

=== PROGRAM ANTRIAN LOKET ===

1. Tambah Antrian
2. Hapus Antrian
3. Tampilkan Antrian
4. Reverse
5. Keluar

4. Contoh Output

=== PROGRAM ANTRIAN LOKET ===

1. Tambah Antrian
2. Hapus Antrian
3. Tampilkan Antrian
4. Reverse
5. Keluar
Pilih menu: 1

Masukkan nama pelanggan: Andi
Data berhasil ditambahkan ke antrian

Pilih menu: 1
Masukkan nama pelanggan: Budi
Data berhasil ditambahkan ke antrian

Pilih menu: 3
Isi antrian:
1. Andi
2. Budi

Pilih menu: 4
Isi antrian:
1. Budi
2. Andi

JAWABAN

1. Kode Program

Berikut dua class kode program untuk tugas pekan 4

1.1 Class AntrianLoket_2511533029

```java
package pekan4_2511533029;
public class AntrianLoket_2511533029 {
    int front_3029;
    int rear_3029;
    int max_3029;
    String queue_3029[];

    public AntrianLoket_2511533029(int max_3029) {
        this.max_3029 = max_3029;
        this.front_3029 = 0;
        this.rear_3029 = -1;
        queue_3029 = new String[this.max_3029];
    }

    boolean isFull_3029(AntrianLoket_2511533029 queue) {
        return (queue.rear_3029 == queue.max_3029 - 1);
    }

    boolean isEmpty_3029(AntrianLoket_2511533029 queue) {
        return (queue.rear_3029 == -1);
    }
    void enqueue_3029(String pelanggan_3029) {
        if (isFull_3029(this)) {
            System.out.println("Antrian penuh");
            return;
        }
        this.rear_3029 = this.rear_3029 + 1;
        this.queue_3029[this.rear_3029] = pelanggan_3029;
        System.out.println(pelanggan_3029 + " berhasil masuk antrian");
    }
    String dequeue_3029() {
        if (isEmpty_3029(this))
            return null;
        String pelanggan_3029 = this.queue_3029[this.front_3029];
        this.front_3029++;

        if (this.front_3029 > this.rear_3029) {
            this.front_3029 = 0;
            this.rear_3029 = -1;
        }
        return pelanggan_3029;
```

Kode Program 1.1

```java
    }
    String front_3029() {
        if (isEmpty_3029(this))
            return null;
        return this.queue_3029[this.front_3029];
    }
    String rear_3029() {
        if (isEmpty_3029(this))
            return null;
        return this.queue_3029[this.rear_3029];
    }
    void display_3029() {
        int i_3029;
        int nomor_3029 = 1;
        if (isEmpty_3029(this)) {
            System.out.println("\nAntrian kosong\n");
            return;
        }
        System.out.println("Isi antrian:");
        for (i_3029 = front_3029; i_3029 <= rear_3029; i_3029++) {
            System.out.println(nomor_3029 + ". " + queue_3029[i_3029]);
            nomor_3029++;
        }
    }
    void reverse_3029() {
        if (isEmpty_3029(this)) {
            System.out.println("\nAntrian kosong\n");
            return;
        }
        int kiri_3029 = front_3029;
        int kanan_3029 = rear_3029;
        while (kiri_3029 < kanan_3029) {
            String temp_3029 = queue_3029[kiri_3029];
            queue_3029[kiri_3029] = queue_3029[kanan_3029];
            queue_3029[kanan_3029] = temp_3029;
            kiri_3029++;
            kanan_3029--;
        }
        System.out.println("Antrian berhasil dibalik");
    }
}
```

Kode Program 1.2

Program AntrianLoket_2511533029 merupakan implementasi struktur data Queue (antrian) menggunakan array dalam bahasa Java. Program ini digunakan untuk mensimulasikan antrian pelanggan pada loket pelayanan dengan prinsip FIFO (First In First Out), yaitu pelanggan yang pertama masuk akan menjadi yang pertama dilayani.

Program memiliki variabel front_3029, rear_3029, max_3029, dan queue_3029[]. Variabel front_3029 menunjukkan posisi data paling depan,

rear_3029 posisi data paling belakang, max_3029 kapasitas maksimum antrian, dan queue_3029[] adalah array bertipe String untuk menyimpan data pelanggan. Constructor digunakan untuk mengatur kapasitas queue, menentukan posisi awal antrian, dan membuat array penyimpanan.

Method isFull_3029() digunakan untuk mengecek apakah antrian penuh, sedangkan isEmpty_3029() digunakan untuk mengecek apakah antrian kosong. Method enqueue_3029() berfungsi menambahkan pelanggan ke bagian belakang antrian, sedangkan dequeue_3029() berfungsi menghapus pelanggan dari bagian depan sesuai urutan FIFO.

Method front_3029() digunakan untuk melihat pelanggan paling depan dan rear_3029() untuk melihat pelanggan paling belakang tanpa menghapus data. Method display_3029() menampilkan seluruh isi antrian secara berurutan, sedangkan reverse_3029() digunakan untuk membalik urutan isi antrian.

1.2 Class Queue_2511533029

```java
package pekan4_2511533029;

import java.util.Scanner;

public class Queue_2511533029 {
    public static void main(String[] args) {
        Scanner input_3029 = new Scanner(System.in);
        AntrianLoket_2511533029 antrian_3029 = new AntrianLoket_2511533029(10);

        int pilihan_3029;
        String pelanggan_3029;

        do {
            System.out.println("\n=== PROGRAM ANTRIAN LOKET ===");
            System.out.println("1. Tambah Antrian");
            System.out.println("2. Hapus Antrian");
            System.out.println("3. Tampilkan Antrian");
            System.out.println("4. Reverse");
            System.out.println("5. Keluar");
            System.out.print("Pilih menu: ");
            pilihan_3029 = input_3029.nextInt();

            switch (pilihan_3029) {
                case 1:
                    System.out.print("Masukkan nama pelanggan: ");
                    pelanggan_3029 = input_3029.next();
                    antrian_3029.enqueue_3029(pelanggan_3029);
                    break;

                case 2:
                    pelanggan_3029 = antrian_3029.dequeue_3029();
                    if (pelanggan_3029 == null) {
                        System.out.println("Antrian kosong");
                    } else {
                        System.out.println("Pelanggan " + pelanggan_3029 + " telah dilayani");
                    }
                    break;
```

Kode Program 1.3

```java

                case 3:
                    antrian_3029.display_3029();
                    break;

                case 4:
                    antrian_3029.reverse_3029();
                    antrian_3029.display_3029();
                    break;

                case 5:
                    System.out.println("Program selesai");
                    break;

                default:
                    System.out.println("Pilihan tidak valid");
            }

        } while (pilihan_3029 != 5);

        input_3029.close();
    }
}
```

Kode Program 1.4

Program Queue_2511533029 merupakan class utama yang digunakan untuk menjalankan program antrian loket. Class ini berfungsi menampilkan menu, menerima input pengguna, dan menjalankan operasi queue dengan memanfaatkan class AntrianLoket_2511533029. Program dimulai dari method main() sebagai titik awal eksekusi.

Pada awal program dibuat objek Scanner bernama input_3029 untuk menerima input dari keyboard, serta objek antrian_3029 dari class AntrianLoket_2511533029 dengan kapasitas maksimal 10 pelanggan. Selain itu, digunakan variabel pilihan_3029 untuk menyimpan pilihan menu dan pelanggan_3029 untuk menyimpan nama pelanggan.

Program menggunakan perulangan do-while agar menu terus ditampilkan sampai pengguna memilih keluar. Menu terdiri dari lima pilihan, yaitu tambah antrian, hapus antrian, tampilkan antrian, reverse, dan keluar. Pilihan pengguna kemudian diproses menggunakan switch-case.

Pada case 1, program meminta nama pelanggan lalu menambahkannya ke antrian dengan method enqueue_3029(). Pada case 2,

program memanggil dequeue_3029() untuk menghapus pelanggan paling depan, lalu menampilkan pelanggan yang telah dilayani atau pesan bahwa antrian kosong. Pada case 3, program memanggil display_3029() untuk menampilkan seluruh isi antrian. Pada case 4, program memanggil reverse_3029() untuk membalik urutan antrian lalu menampilkannya kembali. Pada case 5, program menampilkan pesan bahwa program selesai. Jika input tidak sesuai menu, program menampilkan pesan "Pilihan tidak valid".

Perulangan berhenti saat pengguna memilih menu 5, lalu input_3029.close() digunakan untuk menutup Scanner. Secara keseluruhan, class Queue_2511533029 berfungsi sebagai pengatur alur program yang menghubungkan input pengguna dengan seluruh operasi queue.

2. Output Kode Program

Berikut output dari kode program yang saya buat dan menyesuaikan dengan intruksi tugas.

```
=== PROGRAM ANTRIAN LOKET ===
1. Tambah Antrian
2. Hapus Antrian
3. Tampilkan Antrian
4. Reverse
5. Keluar
Pilih menu: 1
Masukkan nama pelanggan: Andi
Andi berhasil masuk antrian

=== PROGRAM ANTRIAN LOKET ===
1. Tambah Antrian
2. Hapus Antrian
3. Tampilkan Antrian
4. Reverse
5. Keluar
Pilih menu: 1
Masukkan nama pelanggan: Budi
Budi berhasil masuk antrian

=== PROGRAM ANTRIAN LOKET ===
1. Tambah Antrian
2. Hapus Antrian
3. Tampilkan Antrian
4. Reverse
5. Keluar
Pilih menu: 3
Isi antrian:
1. Andi
2. Budi
```

```
=== PROGRAM ANTRIAN LOKET ===
1. Tambah Antrian
2. Hapus Antrian
3. Tampilkan Antrian
4. Reverse
5. Keluar
Pilih menu: 4
Antrian berhasil dibalik
Isi antrian:
1. Budi
2. Andi

=== PROGRAM ANTRIAN LOKET ===
1. Tambah Antrian
2. Hapus Antrian
3. Tampilkan Antrian
4. Reverse
5. Keluar
Pilih menu: 2
Pelanggan Budi telah dilayani

=== PROGRAM ANTRIAN LOKET ===
1. Tambah Antrian
2. Hapus Antrian
3. Tampilkan Antrian
4. Reverse
5. Keluar
Pilih menu: 5
Program selesai
```